

Техническое задание

Безопасный MCP-сервер для интеграции coding agents с KIE API

Версия 1.1 • Архитектура, безопасность, multi-agent интеграция, финансовые guardrails, тестирование и критерии готовности

1. Цель проекта

Необходимо развернуть и адаптировать MCP-сервер для работы локальных coding agents (Codex CLI, Claude Code и Grok Build) с API агрегатора KIE.

- дать coding agents доступ к генерации изображений, редактированию изображений, генерации видео, аудио и другим media-моделям KIE через единый безопасный MCP-интерфейс;
- не привязывать систему к фиксированному набору моделей;
- позволить агенту получать актуальные схемы параметров моделей из документации KIE;
- обеспечить безопасную работу с API-ключом;
- ограничить доступ агента к локальной файловой системе;
- ограничить финансовые риски;
- исключить публичный доступ к MCP;
- сделать архитектуру пригодной для дальнейшего production-использования и интеграции с Python/FastAPI backend.

```
User
  ↓
Codex CLI / Claude Code / Grok Build
  ↓
MCP protocol / stdio
  ↓
Our KIE MCP
  ↓
KIE REST API
  ↓
Image / Video / Audio models
```

На первом этапе MCP должен работать только локально через stdio. Не использовать публичный HTTP MCP, ngrok, Cloudflare Tunnel или открытые порты.

2. Базовый репозиторий

В качестве базы использовать:

```
https://github.com/gugu9999gu/KIE-MCP
```

Не изменять оригинальный upstream напрямую. Создать локальный fork/branch:

```
cd ~/Projects
git clone https://github.com/gugu9999gu/KIE-MCP.git kie-mcp
cd kie-mcp
git checkout -b secure-codex
```

Зафиксировать исходный commit и сохранить hash в SECURITY_AUDIT.md:

3. Основные функциональные требования

- искать актуальную документацию KIE
- получать schema конкретной модели
- создавать generation task
- получать status task
- ожидать завершения task
- получать итоговые URLs
- загружать разрешённые локальные файлы
- загружать изображения по URL
- проверять баланс KIE
- скачивать итоговый результат
- проверять стоимость операции до выполнения
- блокировать потенциально дорогие операции
- работать с новыми моделями без изменения MCP-кода

4. Безопасность API-ключа

API-ключ KIE никогда не должен:

- находиться в Git
- находиться в исходном коде
- попадать в README
- записываться в логи
- возвращаться через MCP response
- передаваться как CLI argument
- попадать в shell history
- отображаться в exception traceback

Для MCP использовать отдельный секрет KIE_MCP_API_KEY. Существующий KIE_API_KEY, используемый coding agents для KIE chat-моделей, не должен использоваться MCP.

```
export KIE_MCP_API_KEY="..." # только локальный dev fallback; не коммитить
```

Код MCP должен получать media-ключ только из выделенного secret source и завершать запуск с понятной ошибкой, если ключ отсутствует. Значение ключа никогда не выводить.

```
os.environ["KIE_MCP_API_KEY"]
```

На macOS предпочтительный production-like вариант — отдельная запись macOS Keychain и launcher/wrapper, который получает секрет в дочерний процесс MCP без записи ключа в Git, CLI arguments или конфигурацию агента. Для production предусмотреть Docker Secret, Kubernetes Secret, Vault или облачный Secrets Manager.

5. Sandbox для локальных файлов

MCP не должен напрямую читать или отправлять в KIE произвольные файлы компьютера. Доступ MCP к локальной файловой системе ограничивается только явно разрешёнными корнями. Это ограничение относится к MCP-интерфейсу; отдельный coding agent с полноценным shell-доступом рассматривается отдельно в threat model.

```
KIE_ALLOWED_UPLOAD_ROOT="$HOME/Projects/kie-workspace/uploads"
KIE_ALLOWED_DOWNLOAD_ROOT="$HOME/Projects/kie-workspace/results"

mkdir -p ~/Projects/kie-workspace/uploads
mkdir -p ~/Projects/kie-workspace/results
```

Разрешённые примеры:

```
~/Projects/kie-workspace/uploads/product.jpg
~/Projects/kie-workspace/uploads/test.webp
```

Запрещённые примеры:

```
~/.ssh/id_rsa
~/.ssh/config
~/.env
~/Documents/*
~/Desktop/*
~/Projects/backend/.env
/etc/passwd
```

6. Защита от path traversal и symlink escape

Перед upload/download необходимо вычислять canonical path через realpath/resolve и проверять, что итоговый путь находится внутри разрешённого корня.

```
resolve(realpath(file))
resolve(realpath(allowed_root))
```

Блокировать ../, ../../, абсолютные пути вне root и symlink escape. Нельзя полагаться только на строковую проверку.

```
uploads/private -> ~/.ssh
upload uploads/private/id_rsa # MUST BE BLOCKED
```

7. Ограничения файлов

```
KIE_MAX_UPLOAD_MB=50
```

До отправки проверять размер файла и отклонять превышение лимита.

7.1 Разрешённые MIME-типы

```
image/jpeg
image/png
image/webp
image/gif
video/mp4
video/webm
video/quicktime
audio/mpeg
audio/wav
audio/mp4
audio/ogg
```

Тип определять не только по расширению, а по MIME/magic bytes. По умолчанию блокировать исполняемые, секретные и архивные форматы (.exe, .sh, .py, .env, .pem, .key, .zip, .tar), если они явно не нужны.

8. Безопасная директория скачивания

Все результаты сохранять только внутри KIE_ALLOWED_DOWNLOAD_ROOT. Агенту нельзя позволять передавать произвольный download_dir.

```
results/task_123/output.webp
```

Запись в /etc, ~/Documents, корень проекта и любые другие директории должна блокироваться.

9. SSRF protection

Для upload_from_url/download_from_url необходимо реализовать SSRF-защиту.

Блокировать localhost, loopback, link-local, metadata endpoints и private ranges:

```
127.0.0.0/8
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
169.254.0.0/16
::1
```

URL по умолчанию должен быть https://. Проверять не только исходный URL, но и redirect target.

10. Network allowlist

Исходящие API-запросы MCP должны выполняться только к известным endpoint KIE.

```
https://api.kie.ai
https://docs.kie.ai
```

Если KIE использует отдельный upload/storage endpoint, добавить его явно. Не позволять агенту задавать base_url через MCP tool input.

11. Запрет произвольного shell execution

Проверить код на использование следующих механизмов:

```
subprocess
os.system
eval
exec
shell=True
Popen
child_process
spawn
execSync
```

MCP не должен запускать произвольные shell-команды на основании параметров агента. Ненужный shell execution удалить.

12. Dependency audit и static analysis

```
uv sync --extra dev
# или
python3 -m venv .venv
source .venv/bin/activate
pip install -e "[dev]"
```

Проверить установленные зависимости и уязвимости:

```
pip list
pip-audit
ruff check .
bandit -r .
```

HIGH/CRITICAL vulnerabilities должны быть исправлены либо документированы с обоснованием.

13. Тестирование безопасности

Запустить существующие тесты:

```
pytest
```

Добавить минимум следующие regression/security tests:

- test_upload_allowed_file
- test_upload_outside_root_blocked
- test_upload_dotdot_blocked
- test_upload_symlink_escape_blocked
- test_upload_too_large_blocked
- test_secret_file_extension_blocked
- test_download_outside_root_blocked
- test_ssrf_localhost_blocked
- test_ssrf_private_ip_blocked
- test_ssrf_metadata_ip_blocked
- test_api_key_not_in_error
- test_api_key_not_in_logs

14. Базовые MCP tools

```
kie_search_docs
kie_get_documentation
kie_create_task
kie_get_task
kie_wait_for_task
kie_get_credits
kie_upload_local_file
kie_upload_from_url
kie_upload_base64
kie_get_download_url
```

Названия можно сохранить из исходного проекта, если они уже реализованы.

15. Dynamic model discovery

Не хардкодить полный список KIE моделей. Агент должен уметь находить новую модель через документацию, получать её schema и создавать task без обновления MCP-кода.

```
kie_search_docs("latest image editing models")
    ↓
kie_get_documentation(...)
    ↓
model identifier + required/optional params + pricing
    ↓
kie_create_task(...)
```

16. High-level MCP tools

16.1 generate_image

```
prompt
model optional
aspect_ratio optional
resolution optional
output_format optional
```

16.2 edit_image

```
image_path OR image_url
prompt
model optional
aspect_ratio optional
resolution optional
```

Локальный файл обязан проходить file security validation.

16.3 generate_video

```
prompt
image_path optional
image_url optional
model optional
duration optional
aspect_ratio optional
resolution optional
```

16.4 remove_background

```
image_path OR image_url
```

16.5 upscale_image

```
image_path OR image_url
scale optional
target_resolution optional
```

17. Product-oriented tool

После базовой стабилизации добавить product_image_create.

```
image_path
product_name optional
style
```

```
aspect_ratio
background
resolution
```

Пример:

```
style=ecommerce
background=white
aspect_ratio=3:4
resolution=4K
```

Workflow:

```
validate input
↓
upload
↓
select suitable KIE model
↓
check schema
↓
calculate cost
↓
create task
↓
wait
↓
download result
```

18. Product pipeline

```
original product photo
↓
cleanup
↓
background removal
↓
white studio background
↓
enhancement
↓
4K upscale
↓
WebP export
```

Каждая операция должна быть самостоятельной task. Не запускать дорогостоящую цепочку автоматически без контроля стоимости.

19. Финансовая безопасность

```
KIE_MAX_TASK_COST_USD=1.00
KIE_MAX_SESSION_COST_USD=5.00
KIE_MAX_DAILY_COST_USD=10.00
```

Перед task получить цену или максимально точно оценить стоимость. Если estimated task cost превышает лимит, отклонить операцию.

```
Generation blocked: estimated cost $X exceeds configured per-task limit $Y.
```

MCP должен вести per-session spend и общий persistent daily spend. При одновременной работе нескольких agents/окон терминала бюджет нельзя считать только в памяти процесса: требуется

общий process-safe usage ledger. Дополнительно использовать KIE-side hourly/daily/total usage caps; их изменяет только владелец вручную. IP whitelist включать только при наличии стабильного доверенного IP.

20. Cost preview и dry-run

20.1 kie_estimate_cost

```
{
  "model": "...",
  "estimated_cost_usd": 0.15,
  "estimated_credits": 15,
  "pricing_source": "live_kie_metadata",
  "confidence": "exact|estimated|unknown"
}
```

20.2 dry_run=true

При dry-run не отправлять generation-запрос. Возвращать выбранную модель, resolved schema, validated input, pricing source, estimated cost/credits, confidence, endpoint и request payload без секретов. Если точную цену определить нельзя, paid execution по умолчанию не запускать автоматически.

21. Secret-safe logging и error sanitization

Разрешено логировать: timestamp, client/agent id, session id, correlation id, tool name, model, task id, status, duration, estimated/actual cost и технические retry-события.

Запрещено логировать: KIE_MCP_API_KEY, KIE_API_KEY, Authorization header, base64 file content, private file contents и raw secrets. Raw prompt логировать только если это явно включено владельцем; по умолчанию хранить hash/короткий безопасный summary.

Перед возвратом ошибок агенту редактировать секреты, Authorization header, sensitive paths и environment values как [REDACTED].

22. Async task handling

KIE работает асинхронно. HTTP 200 означает только успешное создание task, а не завершение генерации.

```
POST createTask
↓
task_id
↓
poll task status
↓
success / fail
↓
result URL
```

23. Polling, retry и concurrency

```
Polling backoff: 2s → 3s → 5s → 8s → 10s → 15s max
KIE_TASK_TIMEOUT_SEC=900
KIE_MAX_CONCURRENT_TASKS=5
KIE_USAGE_LEDGER_PATH=$HOME/.local/share/kie-mcp/usage.db
```

Retry: timeouts, temporary 5xx, 429. Не retry: 400, 401, 403, invalid parameters. Для 429 учитывать Retry-After.

24. Duplicate request protection

Считать fingerprint по model + prompt/input hash + generation params + operation type. Для дорогих операций использовать idempotency/duplicate protection: если идентичный task уже создан и находится в допустимом TTL, возвращать существующий task id, а не создавать дубликат. Результат должен быть безопасен при одновременных процессах.

25. Интеграция с Codex CLI, Claude Code и Grok Build

На первом этапе использовать только local stdio transport. MCP-сервер должен быть client-agnostic и одинаково работать с Codex CLI, Claude Code и Grok Build. Не изменять и не перезаписывать существующие настройки KIE model providers/aliases этих агентов; добавить только отдельную MCP-регистрацию.

```
{
  "example_only": {
    "transport": "stdio",
    "command": "kie-mcp-launch",
    "note": "Конкретный формат регистрации отличается у Codex/Claude/Grok и должен быть описан в README для каждого клиента. Секрет не передавать в args."
  }
}
```

API key не хранить в Git-конфиге и не передавать через CLI arguments. Для dev допускается наследование environment; для рабочего Mac предпочтителен отдельный launcher + macOS Keychain согласно разделу 52.

26. HTTP transport — только позже

На первом этапе HTTP не использовать. Если понадобится позже: host=127.0.0.1, Bearer authentication, Host validation и CORS allowlist. Никогда не слушать 0.0.0.0 без отдельного security review.

27. Версионирование и Git hygiene

Не использовать prx -y latest или любой автоматически обновляющийся внешний код с доступом к API secret. Использовать фиксированный Git commit или фиксированную версию.

Добавить .gitignore минимум:

```
.env
.env.*
.venv/
__pycache__/
```

```
*.pyc
.DS_Store
uploads/
results/
*.log
```

28. Security documentation

Создать SECURITY.md с описанием secret handling, filesystem sandbox, SSRF protection, budget controls, network policy и known limitations.

Создать SECURITY_AUDIT.md со следующей структурой:

```
Repository
Original commit
Audit date
Files inspected
Dependencies
Secrets
Filesystem access
Network access
Command execution
SSRF
Path traversal
Authentication
Logging
Risk findings
Changes made
Remaining risks
```

29. README и .env.example

README должен содержать Installation, Configuration, Codex CLI setup, Claude Code setup, Grok Build setup, Security, Threat Model, Allowed upload directory, Allowed output directory, Budget limits, KIE-side caps, Usage Ledger, Examples, Dry run и Troubleshooting.

```
# KIE MCP (media key only; dev fallback)
KIE_MCP_API_KEY=
KIE_BASE_URL=https://api.kie.ai
KIE_SECRET_SOURCE=env # env|macos_keychain|docker_secret|vault

# File sandbox
KIE_ALLOWED_UPLOAD_ROOT=/Users/USER/Projects/kie-workspace/uploads
KIE_ALLOWED_DOWNLOAD_ROOT=/Users/USER/Projects/kie-workspace/results
KIE_MAX_UPLOAD_MB=50

# Task controls
KIE_TASK_TIMEOUT_SEC=900
KIE_MAX_CONCURRENT_TASKS=5
KIE_USAGE_LEDGER_PATH=/Users/USER/.local/share/kie-mcp/usage.db

# Budget
KIE_MAX_TASK_COST_USD=1.00
KIE_MAX_SESSION_COST_USD=5.00
KIE_MAX_DAILY_COST_USD=10.00

# Paid action policy
KIE_REQUIRE_APPROVAL_IF_COST_UNKNOWN=true
```

30. Первый функциональный тест

1. Выполнить `kie_get_credits`.
2. Выполнить `kie_search_docs`.
3. Выполнить `kie_estimate_cost`.
4. Выполнить `dry_run`.
5. Только после этого выполнить один дешёвый реальный task.

31. Security acceptance tests

```
upload ~/.ssh/id_rsa          → BLOCKED
upload ../../.env             → BLOCKED
symlink escape                → BLOCKED
download outside results root → BLOCKED
URL to localhost/private network → BLOCKED
```

32. Functional image test

```
Input: ~/Projects/kie-workspace/uploads/test.jpg
Prompt: Create a professional clean studio product photo. White background. Preserve the
product exactly.
Expected output: ~/Projects/kie-workspace/results/...
```

33. Definition of Done — Phase 1

- ☐ repository cloned and pinned
- ☐ dedicated branch created
- ☐ dependencies audited
- ☐ tests pass
- ☐ ruff passes
- ☐ security scan completed
- ☐ API key never appears in repository
- ☐ MCP starts over stdio
- ☐ Codex CLI, Claude Code и Grok Build видят одни и те же KIE MCP tools через stdio
- ☐ credits call works
- ☐ documentation search works
- ☐ task creation works
- ☐ task polling works

- ☐ successful generation works
- ☐ local upload sandbox works
- ☐ path traversal blocked
- ☐ symlink escape blocked
- ☐ download sandbox works
- ☐ SSRF protections enabled
- ☐ generation limits implemented
- ☐ dry-run implemented
- ☐ security audit documented

34. Phase 2 — Friendly tools

```
generate_image
edit_image
generate_video
remove_background
upscale_image
product_image_create
```

Агент не должен знать детали API конкретной KIE-модели.

35. Phase 3 — Intelligent model router

```
user request
↓
analyze task
↓
search available KIE models
↓
compare capabilities
↓
compare price
↓
choose appropriate model
```

Model selection должен учитывать task, quality, price, resolution, input format и speed. Не выбирать автоматически самую дорогую модель.

36. Phase 4 — интеграция с SaaS

```
FastAPI
↓
KIE service layer
↓
job queue / Celery
↓
KIE
↓
webhook
```

```
↓  
database  
↓  
object storage
```

KIE MCP и production backend должны использовать общий KIE client module, но разные runtime policies и секреты.

37. Production storage и webhook

Результаты KIE нельзя использовать как permanent storage. После успешной генерации скачать результат, загрузить в собственный S3 / Cloudflare R2 / MinIO и сохранить permanent URL.

Для SaaS перейти с polling на webhook, оставив polling fallback.

```
FastAPI  
↓  
KIE task  
↓  
callback URL  
↓  
FastAPI webhook  
↓  
verify callback  
↓  
save result
```

38. Архитектурный принцип

Не создавать отдельный MCP tool под каждую модель KIE.

```
НЕПРАВИЛЬНО:  
generate_with_kling_26  
generate_with_kling_30  
generate_with_veo_31  
generate_with_qwen  
  
ПРАВИЛЬНО:  
generate_image  
edit_image  
generate_video  
+ internal model router
```

39. Dynamic API principle

При появлении новой модели KIE не переписывать MCP. Система должна discover docs → parse schema → validate parameters → execute task.

40. Ограничения агентной автономии

Coding agent не должен самостоятельно:

- повышать spending limits
- изменять allowed filesystem root
- изменять API endpoint
- отключать security validation

- отключать SSRF protection
- отключать logging redaction

Эти настройки меняются только вручную владельцем системы.

41. Правила выполнения T3 coding-агентом

1. Изучить существующий код.
2. Внести минимальное необходимое изменение.
3. Добавить тест.
4. Запустить тесты.
5. Проверить regressions.

Не переписывать полностью работающий проект без необходимости.

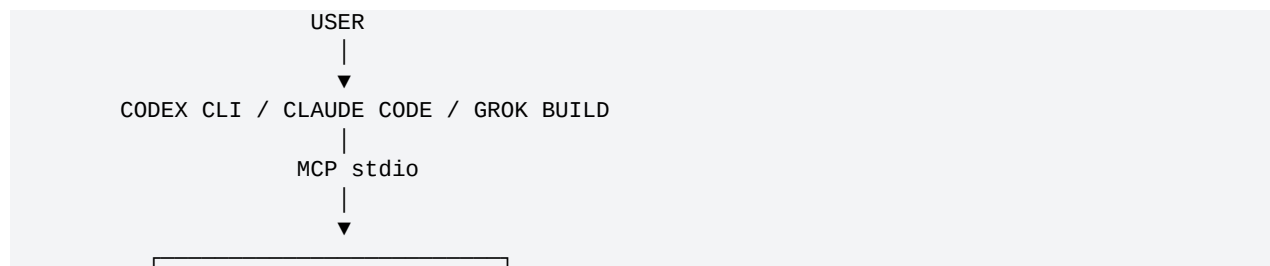
42. Работа с найденными security issues

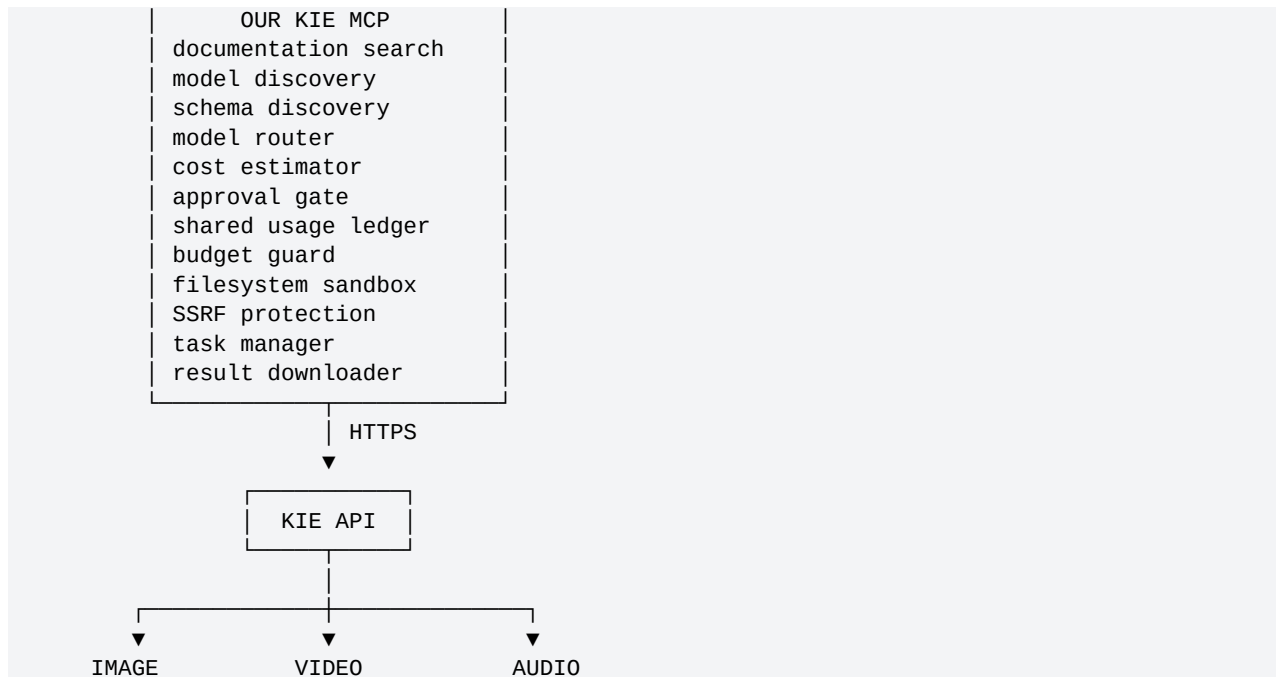
Если найден security issue, агент обязан указать конкретный файл и функцию, объяснить attack scenario, внести исправление и добавить regression test. Формулировки вида «может быть небезопасно» без конкретики не допускаются.

43. Итоговый отчёт агента

- Что было найдено
- Какие security-проблемы были
- Какие исправления внесены
- Какие файлы изменены
- Какие тесты добавлены
- Результат pytest
- Результат ruff
- Результат pip-audit
- Результат bandit
- Как запустить MCP
- Как подключить MCP к Codex CLI, Claude Code и Grok Build
- Как проверить KIE
- Какие риски остаются

44. Финальная целевая архитектура





45. Главный пользовательский сценарий

После выполнения пользователь должен иметь возможность дать любому подключённому coding agent (Codex CLI, Claude Code или Grok Build) команду:

Возьми ~/Projects/kie-workspace/uploads/product.jpg и сделай профессиональную карточку товара. Белый фон. Соотношение 3:4. Высокое качество. Используй наиболее подходящую модель KIE. Перед запуском проверь стоимость.

Agent должен самостоятельно через MCP:

1. проверить файл
2. убедиться, что доступ разрешён
3. найти подходящие KIE модели
4. получить актуальные schemas
5. выбрать модель
6. оценить стоимость
7. проверить budget
8. загрузить файл
9. создать generation task
10. дождаться результата
11. скачать результат
12. сохранить только в разрешённую output directory

13. сообщить model, task ID, стоимость и output path

46. Основной критерий готовности

Через интерфейс этого MCP coding agent не должен иметь возможности:

- напрямую прочитать через MCP секретные файлы компьютера вне разрешённого upload root
- отправить через MCP файл, который не прошёл sandbox/MIME/size/security validation
- записать файлы в произвольные директории
- потратить неограниченное количество KIE credits
- изменить security configuration

Это является основной Definition of Done для самого MCP. Отдельно документировать, что coding agent с независимым shell/network access находится вне полной зоны контроля MCP; для жёсткой изоляции требуется OS/container sandbox.

47. Разделение API-ключей и KIE-side policy

Использовать минимум два независимых KIE API key с разными назначениями:

```
for-mcp-agents / KIE_API_KEY  
→ только coding/chat agents (Codex / Claude / Grok)
```

```
for-mcp-media / KIE_MCP_API_KEY  
→ только KIE MCP media operations
```

- не использовать один и тот же ключ для chat-agent routing и media MCP;
- если KIE UI позволяет ограничить Allowed Models, ключ coding agents ограничить chat/coding моделями, а media key — media-моделями;
- для media key обязательно задать KIE-side hourly, daily и total credit caps; значения являются deployment policy и меняются только владельцем вручную;
- не реализовывать MCP tool для изменения KIE-side limits, allowed models, IP whitelist или ключей;
- IP whitelist включать только для стабильного IP/сервера; для ноутбука с меняющимся IP оставить выключенным до отдельного решения.

48. Multi-agent и multi-process режим

Система должна корректно работать, когда одновременно открыты несколько окон терминала и разные agents используют один и тот же MCP. Каждый client может запустить собственный stdio MCP process, поэтому состояние, влияющее на деньги и idempotency, нельзя хранить только в RAM процесса.

- Codex CLI, Claude Code и Grok Build должны использовать одинаковую семантику tools;
- каждый MCP process получает собственный process/session id;
- clientInfo из MCP initialize использовать как agent identity, если библиотека предоставляет его; иначе формировать безопасный fallback id;
- общие daily/total budgets и duplicate protection должны быть согласованы между процессами;
- никакой процесс не должен иметь возможность повышать общие лимиты.

49. Persistent Usage Ledger

Добавить общий persistent ledger, по умолчанию SQLite, доступный всем локальным MCP-процессам.

```
KIE_USAGE_LEDGER_PATH="$HOME/.local/share/kie-mcp/usage.db"  
SQLite: WAL mode + busy_timeout + transactions
```

Минимальные поля ledger:

```
id  
created_at  
client_name  
client_version  
agent_session_id  
mcp_process_id  
correlation_id  
operation  
model  
provider_task_id  
fingerprint  
status  
estimated_cost_usd  
estimated_credits  
actual_cost_usd  
actual_credits  
pricing_source  
input_hash  
output_count
```

- бюджетные проверки и reservation должны выполняться транзакционно;
- добавить reservation перед paid request и reconcile после завершения/ошибки;
- на crash/timeout зависшие reservations должны иметь безопасный reconciliation/expiry policy;
- добавить regression test на race condition: два процесса одновременно не могут оба пройти один и тот же оставшийся budget.

50. Двухфазный запуск платных операций

Для paid generation использовать prepare/execute flow, чтобы агент не мог незаметно изменить модель или параметры между оценкой стоимости и запуском.

```
Phase A: prepare  
  validate input  
  resolve model/schema  
  estimate cost  
  reserve budget  
  return approval_id + immutable request digest
```

```
Phase B: execute  
  require approval_id  
  verify digest/TTL/budget reservation  
  create KIE task  
  reconcile actual usage
```

- если cost неизвестна или confidence=unknown — execution блокировать до явного разрешения владельца;
- если параметры после prepare изменились — approval_id недействителен;
- dry_run не создаёт reservation и никогда не вызывает платный endpoint;
- никакой retry 400/401/403/invalid params не должен создавать новый paid task.

51. Политика против обхода MCP guardrails

На машине могут быть установлены KIE skills, включая kie-models. После ввода MCP в эксплуатацию media generation должна идти через безопасный MCP path, иначе filesystem/budget guardrails можно обойти.

- kie-chat-agents можно использовать для выбора/настройки KIE chat models;
- kie-models не должен использоваться для прямых paid media generations в production workflow;
- в AGENTS.md / CLAUDE.md / Grok instructions явно указать: media operations выполняются только через KIE MCP;
- наиболее надёжная защита — model allowlist на KIE keys: coding-agent key не должен иметь media models;
- не считать текстовую инструкцию агенту полноценной security boundary; enforcement должен быть на ключах, MCP и OS/container уровне.

52. Secret loading: macOS Keychain и launcher

Для рабочего Mac предпочтительно не экспортировать KIE_MCP_API_KEY глобально в ~/.zshrc. Реализовать launcher `kie-mcp-launch`, который получает media key из macOS Keychain и передаёт его только дочернему MCP process.

```
Coding Agent
  ↓ stdio spawn
kie-mcp-launch
  ↓ read secret from macOS Keychain
export KIE_MCP_API_KEY=<secret in child env>
  ↓ exec
kie-mcp
```

- не печатать результат `security find-generic-password`;
- не передавать secret в argv;
- не писать secret в .env рабочего репозитория;
- environment fallback разрешить только для dev/tests;
- добавить тесты redaction для Keychain/env errors.

53. Threat model и границы защиты

Документировать явную границу: filesystem sandbox MCP гарантирует, что сам MCP не откроет path вне allowlist. Он не способен полностью остановить отдельный coding agent, если тот одновременно имеет произвольный shell/network access от имени того же OS user.

Для сильной изоляции предусмотреть режим hard sandbox:

- отдельный OS user или контейнер для MCP;
- read-only filesystem за исключением разрешённых upload/results mounts;
- media API key доступен только MCP runtime;
- network egress разрешён только к утверждённым KIE endpoints;
- никаких mounts ~/.ssh, home, project secrets или Docker socket.

Phase 1 local stdio считается defense-in-depth. Hard sandbox является рекомендуемым шагом перед production/широкой автономией agents.

54. Усиление SSRF, DNS и redirect validation

- проверять hostname и все resolved A/AAAA addresses до запроса;
- отклонять hostname, если хотя бы один resolved address попадает в blocked/private/link-local/loopback/metadata ranges;
- повторять проверку после каждого redirect;
- ограничить число redirects;
- не доверять только строке hostname; учитывать DNS rebinding/смену resolution;
- добавить IPv6 private/special ranges в security implementation;
- для download/upload-from-url применять timeout, max bytes и streaming.

55. File TOCTOU, uploads и downloads

- после canonical-path validation открывать файл безопасным способом, минимизируя symlink TOCTOU; при поддержке платформы использовать O_NOFOLLOW/эквивалент;
- после открытия проверить fstat/size и при необходимости hash; не доверять только pre-open stat;
- base64 input ограничивать по decoded byte size до декодирования/сразу при streaming;
- скачивания писать во временный файл внутри allowed root, затем fsync/atomic rename;
- не перезаписывать существующий result без явной безопасной политики;
- каждый task сохранять в results/<task_id>/ с server-generated filename, а не caller-controlled path.

56. Pricing correctness и credits

Не использовать глобальное предположение, что KIE credit всегда равен фиксированной сумме USD для всех media-моделей. Стоимость должна определяться из live KIE pricing/model metadata для конкретной операции.

- kie_estimate_cost обязан возвращать источник цены и confidence;
- если pricing зависит от duration/resolution/quality/input type — все эти параметры должны входить в расчёт;
- после completion сохранять actual credits/cost, если KIE их возвращает или их можно надёжно получить из usage API;
- при расхождении estimate/actual ledger должен хранить обе величины;
- KIE-side credit caps рассматриваются как дополнительный hard stop, а не замена локальному cost guard.

57. Интеграция с существующими настройками coding agents

На машине уже могут существовать отдельные providers/aliases для KIE и Experiential. Реализация MCP не должна их переписывать.

Examples of existing independent model routing:
Codex → KIE or Experiential
Claude Code → KIE
Grok Build → KIE

MCP remains independent:
Any supported coding agent → stdio → KIE MCP → KIE media API

- добавлять MCP registration idempotently;
- перед изменением client config делать backup;
- не менять default model/model_provider агента;

- не менять существующие aliases shell;
- README должен показывать install/uninstall/verify для Codex CLI, Claude Code и Grok Build отдельно.

58. Дополнительные security/regression tests v1.1

```
test_separate_media_key_required
test_chat_key_not_used_by_mcp
test_secret_not_in_argv
test_secret_not_in_child_error
test_multi_process_daily_budget_atomic
test_budget_reservation_race
test_stale_reservation_reconciliation
test_prepare_execute_digest_mismatch_blocked
test_unknown_cost_requires_approval
test_duplicate_paid_request_idempotent
test_dns_private_resolution_blocked
test_redirect_to_private_ip_blocked
test_ipv6_private_range_blocked
test_symlink_swap_toctou_blocked_or_detected
test_download_atomic_inside_root
test_result_filename_not_caller_controlled
test_client_config_update_is_idempotent
```

59. Definition of Done — v1.1 additions

- отдельный KIE media key используется только MCP;
- KIE-side spend caps настроены владельцем и MCP не может их менять;
- Codex CLI, Claude Code и Grok Build могут вызвать одинаковый MCP toolset;
- multi-process ledger и budget reservation проходят concurrency tests;
- paid operations используют prepare/execute или эквивалентный immutable approval gate;
- unknown pricing не приводит к автоматическому paid execution;
- MCP secret не хранится в репозитории, argv или agent config;
- документирован threat model и limitation shell-enabled agents;
- hard-sandbox deployment path описан;
- direct media path через agent key технически ограничен где возможно;
- все security tests v1.0 + v1.1 проходят;
- финальный SECURITY_AUDIT.md содержит remaining risks и конкретные mitigations.